BVRIT HYDERABAD College of Engineering for Women

Roll No: 23WH1A0520

1. Title: Node.js Environment Setup for Mentor-Mentee Backend Server

Page No: _____

Date:

Aim:

To configure the Node.js runtime environment and initialize the backend server required for the Mentor-Mentee System using Express.js and MongoDB connectivity.

Description:

In this experiment, the backend environment of the Mentor-Mentee System is prepared using Node.js. The server is initialized with Express.js to handle API communication between the frontend and backend. The database connection is established using Mongoose so that mentor, mentee, session, goal, conversation, and notification data can be stored in MongoDB. The application is structured in a modular form, where configuration files, models, controllers, routes, middleware, and services are maintained separately.

The environment setup is an essential first step because the remaining experiments depend on a stable server configuration. The backend also enables cross-origin requests from the React application and starts a Socket.io server for real-time messaging between mentors and mentees.

Source code:

```
// Backend/src/config/db.js
import mongoose from "mongoose";
import { env } from "./env.js";

export const connectDB = async () => {
  await mongoose.connect(env.mongoUri);
  console.log("MongoDB connected");
};

// Backend/src/server.js
import http from "http";
import { Server } from "socket.io";
import app from "./app.js";
import { connectDB } from "./config/db.js";
import { env } from "./config/env.js";
import { initializeSocket } from "./sockets/socketServer.js";

const startServer = async () => {
  await connectDB();

  const server = http.createServer(app);
  const io = new Server(server, {
    cors: {
      origin: env.clientUrl,
```

```
  });

  initializeSocket(io);

  server.listen(env.port, () => {
    console.log(`Server listening on port ${env.port}`);
  });
};

startServer();
```

Output:

```
> npm run dev
MongoDB connected
Server listening on port 5000
```

Aim:

To implement secure user registration and login functionality for mentors and mentees using Express.js, MongoDB, bcrypt password hashing, and JWT-based authentication.

Description:

This experiment focuses on authentication in the Mentor-Mentee System. New users can register either as mentors or mentees. During registration, user credentials are stored in MongoDB, and passwords are hashed using bcrypt before saving. Depending on the selected role, a corresponding mentor profile or mentee profile is also created.

During login, the system verifies the email and password, checks whether the account is blocked, and then returns a JSON Web Token. This token is used in subsequent protected routes so that only authenticated users can access dashboards, request management, sessions, goals, messages, and notifications. This experiment establishes the foundation for role-based access control in the application.

Source code:

```
// Backend/src/models/User.js
import mongoose from "mongoose";
import bcrypt from "bcryptjs";

const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, minlength: 6 },
  role: { type: String, enum: ["admin", "mentor", "mentee"], required: true },
  isBlocked: { type: Boolean, default: false },
}, { timestamps: true });

userSchema.pre("save", async function save(next) {
  if (!this.isModified("password")) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});

userSchema.methods.comparePassword = function comparePassword(password) {
  return bcrypt.compare(password, this.password);
};

export default mongoose.model("User", userSchema);

// Backend/src/controllers/authController.js
export const register = asyncHandler(async (req, res) => {
```

```
  sendResponse(res, {
    statusCode: 201,
    message: "Registration successful",
    data: buildAuthPayload(user),
  });
});

export const login = asyncHandler(async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });

  if (!user || !(await user.comparePassword(password))) {
    throw new ApiError(401, "Invalid credentials");
  }

  sendResponse(res, {
    message: "Login successful",
    data: buildAuthPayload(user),
  });
});
```

Output:

```
Registration Response:
{
  "success": true,
  "message": "Registration successful",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "_id": "6616c1b1258b12c3c98ab111",
      "name": "Riya Sharma",
      "email": "riya@mentorapp.com",
      "role": "mentee"
    }
  }
}

Login Response:
{
  "success": true,
  "message": "Login successful"
}
```

Aim:

To perform file operations in Node.js for storing and retrieving backup mentorship data such as sample requests, session summaries, and export logs.

Description:

In addition to database storage, file handling is useful for creating local backups, exporting reports, and maintaining seed data or summary logs. In this experiment, the Node.js `fs` module is used to write mentorship data into a JSON file and read it back when required. This kind of utility is valuable during development, testing, and report generation.

For the Mentor-Mentee System, file operations can be used to store a daily snapshot of requests and sessions. It ensures that sample data can be preserved separately from the live database and reviewed later for debugging or demonstration purposes.

Source code:

```
import fs from "fs/promises";
import path from "path";

const exportPath = path.join(process.cwd(), "exports", "mentorship-summary.json");

const mentorshipSummary = {
  generatedOn: new Date().toISOString(),
  requests: [
    { mentee: "Riya Sharma", mentor: "Aarav Rao", status: "accepted" },
    { mentee: "Dev Kumar", mentor: "Priya Nair", status: "pending" }
  ],
  sessions: [
    { topic: "Resume Review", date: "2026-04-15", status: "confirmed" },
    { topic: "DSA Guidance", date: "2026-04-18", status: "pending" }
  ]
};

export async function storeMentorshipSummary() {
  await fs.mkdir(path.dirname(exportPath), { recursive: true });
  await fs.writeFile(exportPath, JSON.stringify(mentorshipSummary, null, 2),
"utf8");
  console.log("Mentorship summary stored successfully");
}

export async function readMentorshipSummary() {
  const data = await fs.readFile(exportPath, "utf8");
  const parsed = JSON.parse(data);
  console.log("Stored requests:", parsed.requests.length);
  console.log("Stored sessions:", parsed.sessions.length);
```

Output:

```
Mentorship summary stored successfully
Stored requests: 2
Stored sessions: 2
File created: exports/mentorship-summary.json
```

Aim:

To build Express.js routes and controllers for creating, viewing, and updating mentorship requests in the Mentor-Mentee System.

Description:

This experiment demonstrates how the Express server handles mentorship request workflows. A mentee can send a request to a mentor by including a message, goals, and preferred time. The server checks whether a pending request already exists and, if not, stores the new request in MongoDB. The system also allows mentors and mentees to fetch request lists relevant to them.

When a mentor accepts a request, the system updates the request status, increases the mentor's guided mentee count, creates a conversation channel if required, and sends a notification to the mentee. This experiment reflects the core interaction model of the Mentor-Mentee System.

Source code:

```
// Backend/src/routes/requestRoutes.js
import { Router } from "express";
import { protect } from "../middleware/authMiddleware.js";
import {
  createRequest,
  getRequests,
  updateRequestStatus,
} from "../controllers/requestController.js";

const router = Router();

router.get("/", protect, getRequests);
router.post("/", protect, createRequest);
router.patch("/:id", protect, updateRequestStatus);

export default router;

// Backend/src/controllers/requestController.js
export const createRequest = asyncHandler(async (req, res) => {
  const { mentorId, message, goals, preferredTime } = req.body;

  const request = await MentorshipRequest.create({
    mentorId,
    menteeId: req.user._id,
    message,
    goals,
    preferredTime,
  });
```

```
    title: "New mentorship request",
    message: "A mentee has sent you a mentorship request.",
    type: "request",
  });

  sendResponse(res, { statusCode: 201, message: "Request sent", data: request });
});
```

Output:

```
POST /api/requests
Status: 201 Created

{
  "success": true,
  "message": "Request sent",
  "data": {
    "_id": "6616c410258b12c3c98ab222",
    "mentorId": "6616c1b1258b12c3c98ab101",
    "menteeId": "6616c1b1258b12c3c98ab111",
    "message": "I need guidance for placements and DSA.",
    "goals": ["Resume review", "Mock interview"],
    "preferredTime": "Evening",
    "status": "pending"
  }
}
```

Aim:

To design MongoDB collections and schemas required for storing all major entities of the Mentor-Mentee System.

Description:

The Mentor-Mentee System requires a well-organized database design to manage multiple entities. MongoDB is used as the database, and Mongoose schemas define the structure of each collection. Separate models are maintained for users, mentor profiles, mentee profiles, mentorship requests, sessions, goals, conversations, messages, and notifications.

This schema-based approach improves maintainability, supports validation, and makes relationships clear through ObjectId references. The database design also supports role-based workflows, accepted mentor-mentee pairing, session scheduling, progress tracking, and communication history.

Source code:

```
// Backend/src/models/MentorshipRequest.js
import mongoose from "mongoose";

const mentorshipRequestSchema = new mongoose.Schema({
  menteeId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  mentorId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  message: { type: String, required: true },
  goals: [{ type: String }],
  preferredTime: { type: String, default: "" },
  status: {
    type: String,
    enum: ["pending", "accepted", "rejected"],
    default: "pending",
  },
}, { timestamps: true });

export default mongoose.model("MentorshipRequest", mentorshipRequestSchema);

// Backend/src/models/Session.js
const sessionSchema = new mongoose.Schema({
  mentorId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  menteeId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  date: { type: String, required: true },
  startTime: { type: String, required: true },
  endTime: { type: String, required: true },
  topic: { type: String, required: true },
  meetLink: { type: String, default: "" },
  status: {
    type: String,
```

```
  },
}, { timestamps: true });

// Backend/src/models/Goal.js
const goalSchema = new mongoose.Schema({
  menteeId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  mentorId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  title: { type: String, required: true },
  progress: { type: Number, default: 0, min: 0, max: 100 },
  status: {
    type: String,
    enum: ["not-started", "in-progress", "completed"],
    default: "not-started",
  },
}, { timestamps: true });
```

Output:

```
MongoDB Collections Created:
1. users
2. mentorprofiles
3. menteeprofiles
4. mentorshiprequests
5. sessions
6. goals
7. conversations
8. messages
9. notifications

Console Output:
MongoDB connected
```

Aim:

To perform Create, Read, Update, and Delete operations on key collections such as users, mentorship requests, and sessions using Mongoose.

Description:

CRUD operations form the basic data management layer of the application. In the Mentor-Mentee System, users are created during registration, requests are created when a mentee applies to a mentor, sessions are created when meetings are scheduled, and records are updated whenever statuses or profile information change.

This experiment uses Mongoose methods such as `create`, `find`, `findByIdAndUpdate`, and `findByIdAndDelete`. These operations help the application manage the complete life cycle of mentorship data in a structured and efficient way.

Source code:

```
import User from "../models/User.js";
import MentorshipRequest from "../models/MentorshipRequest.js";
import Session from "../models/Session.js";

export async function createMentorshipRequest(payload) {
  return MentorshipRequest.create(payload);
}

export async function getMenteeRequests(menteeId) {
  return MentorshipRequest.find({ menteeId }).populate("mentorId", "name email");
}

export async function updateSessionStatus(sessionId, status) {
  return Session.findByIdAndUpdate(
    sessionId,
    { status },
    { new: true }
  );
}

export async function deleteUser(userId) {
  return User.findByIdAndDelete(userId);
}
```

Output:

```
Create Operation:
Request inserted successfully.
```

```
Update Operation:
Session status changed from "pending" to "confirmed".

Delete Operation:
Selected user record deleted successfully.
```

Aim:

To implement MongoDB query operations that filter mentors and system records, sort results, count matching documents, and limit displayed data for dashboards.

Description:

In real applications, fetching every record is inefficient. Therefore, query operations are required to retrieve only meaningful data. In the Mentor-Mentee System, mentors can be filtered by category, experience, approval status, or search keyword. Requests and sessions can be sorted by creation date, and the dashboard can show only a limited number of recent entries.

MongoDB query functions like `find`, `sort`, `limit`, and `countDocuments` are highly useful in such cases. This experiment demonstrates how the system produces optimized and user-specific results for dashboard and listing pages.

Source code:

```
import MentorProfile from "../models/MentorProfile.js";
import Session from "../models/Session.js";
import MentorshipRequest from "../models/MentorshipRequest.js";

export async function getFilteredMentors(searchText) {
  return MentorProfile.find({
    isApproved: true,
    skills: { $regex: searchText, $options: "i" },
  })
    .populate("userId", "name email")
    .sort({ totalMenteesGuided: -1 })
    .limit(5);
}

export async function countPendingRequests(mentorId) {
  return MentorshipRequest.countDocuments({
    mentorId,
    status: "pending",
  });
}

export async function getRecentSessions(userId) {
  return Session.find({
    $or: [{ mentorId: userId }, { menteeId: userId }],
  }).sort({ createdAt: -1 }).limit(10);
}
```

Output:

BVRIT HYDERABAD College of Engineering for Women

```
Count Result:
Pending requests for mentor: 4

Sorted and Limited Result:
10 recent sessions displayed in descending order of creation date.
```

Aim:

To create a React form interface that allows mentees to submit mentorship requests and schedule sessions with mentors.

Description:

Frontend forms are used to collect structured input from the user. In this experiment, a React form is designed for scheduling a mentorship session. The form captures mentor selection, date, time, topic, meeting link, and notes. Controlled components are used so that React state always reflects the current field values.

When the user submits the form, the data is sent to the backend using Axios through a dedicated service layer. This experiment improves user interaction and provides a practical interface for initiating mentorship activities.

Source code:

```
import { useState } from "react";
import toast from "react-hot-toast";
import { dataService } from "../../services/dataService";

export default function SessionForm() {
  const [bookingForm, setBookingForm] = useState({
    mentorId: "",
    date: "",
    startTime: "",
    endTime: "",
    topic: "",
    meetLink: "",
    notes: "",
  });

  const handleBooking = async () => {
    try {
      await dataService.createSession({
        ...bookingForm,
        status: "pending",
      });
      toast.success("Session request sent");
    } catch (error) {
      toast.error(error.message || "Unable to book session");
    }
  };

  return (
    <button type="button" onClick={handleBooking}>
```

```
  );
}
```

Output:

```
Frontend Output:
A session creation form is displayed with fields for mentor, date, start time, end
time, topic, meeting link, and notes.

On Submit:
"Session request sent"
```

Aim:

To validate user input in the Mentor-Mentee System using frontend checks and backend request validation.

Description:

Validation is required to ensure that only correct and complete data enters the system. In this experiment, form fields such as name, email, password, topic, and mentor selection are validated before processing. On the frontend, checks prevent empty or invalid values from being submitted. On the backend, Express validation middleware captures invalid payloads and returns descriptive error responses.

This experiment improves reliability, user experience, and database integrity. It also reduces the possibility of malformed input disrupting the mentorship workflow.

Source code:

```
// Frontend validation example
const validateSessionForm = (form) => {
  if (!form.mentorId) return "Select a mentor first";
  if (!form.date) return "Date is required";
  if (!form.startTime || !form.endTime) return "Session timing is required";
  if (!form.topic.trim()) return "Topic is required";
  return "";
};

// Backend/src/middleware/validateRequest.js
import { validationResult } from "express-validator";
import { ApiError } from "../utils/ApiError.js";

export const validateRequest = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return next(new ApiError(400, "Validation failed", errors.array()));
  }
  next();
};
```

Output:

```
Sample Validation Messages:
1. Select a mentor first
2. Date is required
3. Topic is required

Backend Error Response:
```

```
  "message": "Validation failed"
}
```

Aim:

To retrieve mentor, mentee, and session data from the backend and render it dynamically in React components.

Description:

Dynamic data rendering is an important part of a MERN stack application. In this experiment, React components use service-layer API calls to fetch mentors, requests, and sessions from the Express backend. A custom fetch hook handles loading state and response data. The retrieved data is then rendered in cards and sections for the mentee and mentor dashboards.

This experiment demonstrates how React works with backend APIs to show live project data instead of static content. It also improves reusability by centralizing network requests in service modules.

Source code:

```
import { Loader } from "../../components/common/Loader";
import { useFetch } from "../../hooks/useFetch";
import { dataService } from "../../services/dataService";

export default function MenteeRequestsPage() {
  const { data, loading } = useFetch(() => dataService.getRequests(), []);

  if (loading) return <Loader />;

  return (
    <div>
      {(data || []).map((request) => (
        <div key={request._id}>
          <h3>{request.mentorId?.name}</h3>
          <p>{request.message}</p>
          <span>{request.status}</span>
        </div>
      ))}
    </div>
  );
}
```

Output:

```
UI Output:
Mentor Name: Aarav Rao
Message: I need guidance for placements and DSA.
Status: accepted

Additional Result:
```

BVRIT HYDERABAD College of Engineering for Women

Aim:

To integrate the React frontend with the Node.js and Express backend so that the Mentor-Mentee System works as a complete full-stack application connected to MongoDB.

Description:

This experiment demonstrates end-to-end integration across the MERN stack. The React frontend uses Axios to send API requests, the Express server processes the request, controllers interact with MongoDB using Mongoose, and the result is returned as JSON. Authentication tokens are automatically attached through Axios interceptors for secure access.

This integration is the core of the project because it allows registration, login, mentorship requests, session scheduling, goal tracking, and notifications to function as one connected system. It also ensures separation of concerns between presentation, routing, business logic, and persistence layers.

Source code:

```
// Frontend/src/services/api.js
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:5000/api",
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("mentor_token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default api;

// Frontend/src/services/dataService.js
export const dataService = {
  getRequests: () => api.get("/requests"),
  createRequest: (payload) => api.post("/requests", payload),
  getSessions: () => api.get("/sessions"),
  createSession: (payload) => api.post("/sessions", payload),
};
```

Output:

```
Integration Flow:
React Form -> Axios Request -> Express Route -> Controller -> MongoDB -> JSON
```

```
Sample Response:
{
  "success": true,
  "message": "Session created"
}
```

Aim:

To implement a notification system that informs mentors and mentees about new requests, session schedules, and other important updates.

Description:

Notifications help users remain informed about important system events. In the Mentor-Mentee System, notifications are generated when a new mentorship request is sent, when request status changes, when a session is scheduled or updated, and when goals are modified. Notifications are stored in MongoDB and displayed on a dedicated notifications page in the frontend.

This experiment improves user engagement and ensures that mentors and mentees do not miss important events. The notification model also provides support for read and unread status management.

Source code:

```
// Backend/src/services/notificationService.js
import Notification from "../models/Notification.js";

export const createNotification = async ({ userId, title, message, type }) =>
  Notification.create({ userId, title, message, type });

// Backend/src/controllers/notificationController.js
export const getNotifications = asyncHandler(async (req, res) => {
  const notifications = await Notification.find({ userId: req.user._id })
    .sort({ createdAt: -1 });
  sendResponse(res, { data: notifications });
});

export const markNotificationRead = asyncHandler(async (req, res) => {
  const notification = await Notification.findByIdAndUpdate(
    { _id: req.params.id, userId: req.user._id },
    { isRead: true },
    { new: true }
  );
  sendResponse(res, { message: "Notification updated", data: notification });
});
```

Output:

```
Notification List:
1. New mentorship request
2. Mentorship request updated
3. Session scheduled
4. Goal updated
```

BVRIT HYDERABAD College of Engineering for Women

Aim:

To track the changing status of mentorship requests, sessions, and goals throughout the application workflow.

Description:

Status tracking is essential for understanding progress in a mentor-mentee platform. In this project, requests can be pending, accepted, or rejected. Sessions can be pending, confirmed, completed, or cancelled. Goals can be not-started, in-progress, or completed. These statuses are stored in MongoDB and updated by mentors or mentees depending on role permissions.

This experiment helps the users monitor the progress of their interactions clearly. It also supports transparent communication between mentors and mentees by keeping request, meeting, and learning progress visible in the dashboard.

Source code:

```
// Request status update
request.status = req.body.status;
await request.save();

// Session status schema
status: {
  type: String,
  enum: ["pending", "confirmed", "completed", "cancelled"],
  default: "pending",
}

// Goal status schema
status: {
  type: String,
  enum: ["not-started", "in-progress", "completed"],
  default: "not-started",
}

// Goal update controller
export const updateGoal = asyncHandler(async (req, res) => {
  const goal = await Goal.findByIdAndUpdate(req.params.id, req.body, { new: true
});
  sendResponse(res, { message: "Goal updated", data: goal });
});
```

Output:

```
Sample Status Values:
Request Status: accepted
```

BVRIT HYDERABAD College of Engineering for Women

Dashboard Result:
Updated badges are shown immediately after the record is changed.

Aim:

To implement page navigation and role-based routing in the React frontend using React Router.

Description:

Navigation in the Mentor-Mentee System is handled through React Router. Public routes are used for landing, about, contact, login, register, and mentor browsing pages. Protected routes are used for mentee, mentor, and admin dashboards. Each role sees only the pages relevant to its workflow.

This experiment ensures structured and secure navigation throughout the project. It also improves user experience by organizing the application into meaningful sections such as requests, sessions, goals, messages, notifications, approvals, and reports.

Source code:

```
import { Navigate, Route, Routes } from "react-router-dom";
import { ProtectedRoute } from "./ProtectedRoute";

export function AppRoutes() {
  return (
    <Routes>
      <Route path="/" element={<LandingPage />} />
      <Route path="/login" element={<LoginPage />} />
      <Route path="/register" element={<RegisterPage />} />

      <Route element={<ProtectedRoute roles={["mentee", "mentor", "admin"]} />}>
        <Route element={<DashboardLayout />}>
          <Route element={<ProtectedRoute roles={["mentee"]} />}>
            <Route path="/mentee/requests" element={<MenteeRequestsPage />} />
            <Route path="/mentee/sessions" element={<MenteeSessionsPage />} />
          </Route>

          <Route element={<ProtectedRoute roles={["mentor"]} />}>
            <Route path="/mentor/requests" element={<MentorRequestsPage />} />
            <Route path="/mentor/messages" element={<MentorMessagesPage />} />
          </Route>

          <Route element={<ProtectedRoute roles={["admin"]} />}>
            <Route path="/admin/users" element={<AdminUsersPage />} />
          </Route>
        </Route>
      </Route>

      <Route path="*" element={<Navigate to="/" replace />} />
    </Routes>
```

Output:

```
Navigation Result:
1. Public users can open landing, login, register, and mentor listing pages.
2. Mentees can access requests, sessions, goals, messages, and notifications pages.
3. Mentors can access mentees, requests, sessions, and messages pages.
4. Admin can access dashboard, users, approvals, categories, and reports pages.
```